

0.1. Afficher la lettre A

Une seule lettre, dans la toute première case de l'écran.

1. Ce qu'il faut comprendre

Les lettres sont **déjà dessinées** dans le programme : c'est la police, chargée au démarrage. Il suffit de dire laquelle afficher, et où.

`texte(0, 0, "A")` écrit **A** dans la case de la colonne **0**, ligne **0** : celle en haut à gauche. L'écran fait 20 colonnes (0 à 19) sur 18 lignes (0 à 17).

La ligne est écrite **avant** la boucle : on n'écrit le A qu'une fois, et il reste affiché.

La première ligne, `#include <texte>`, dit à la console : « ce programme se sert de `texte()` ». Aucune fonction de la console n'est là d'office : chacune prend de la **place dans la cartouche** (son code, ses lettres), et ne s'emploie que si on l'**inclut**, par son nom. Sans cette ligne, le compilateur refuse `texte()`, et dit quelle ligne écrire.

Au 0.0, il n'y avait aucun `#include` : `image()` ne coûte rien, elle est toujours là. À partir d'ici, chaque programme commence par la liste de ce qu'il emploie — une ligne par fonction, avec en commentaire ce qu'elle fait.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction `texte\(\)` — écrire un mot](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **203 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage	— la mise en route de la console
main()	18 o	← démarrage	— ta fonction
VBlank	8 o	← la console, à chaque image	— compte les images, 60 fois par seconde
AttendreImage	28 o	← <code>image()</code>	— attend l'image suivante
AttendreVBlank	16 o	← démarrage	— attend que l'écran soit libre
AttendreAcces	12 o	← <code>EcrireTexte</code>	— attend de pouvoir écrire une case
EcrireTexte	10 o	← <code>texte()</code>	— écrit un texte, lettre par lettre
CopierTuiles	31 o	← démarrage	— copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage	— vide les deux cartes de l'écran
Tuiles	16 o	← <code>CopierTuiles</code>	— 1 lettre sur 43
"A"	1 o	← <code>texte()</code>	— un texte gravé

Lettres gravées : A

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `AdresseCase`, `EffacerCases`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

<code>texte()</code> × 1	11 o sur place, dans <code>main()</code>	+ <code>EcrireTexte</code> (10 o)
<code>image()</code> × 1	3 o sur place, dans <code>main()</code>	+ <code>AttendreImage</code> (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

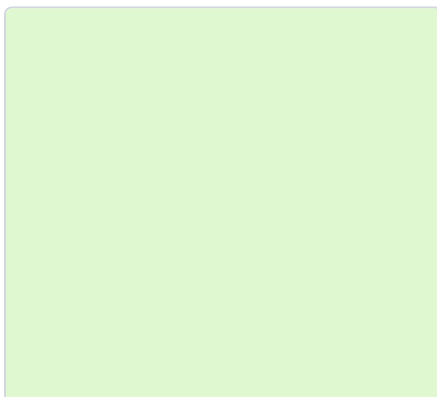
```
// #include <texte> : « ce programme se sert de texte() ».  
// Une fonction de la console prend de la place dans la cartouche :  
// elle ne s'emploie que si on l'inclut, par son nom. Sans cette ligne,  
// le compilateur refuse texte() – et dit quelle ligne écrire.  
#include <texte> // écrit un texte à l'écran
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() { // Le programme commence ici.  
    texte(0, 0, "A"); // colonne 0, ligne 0 : la case en haut à gauche  
                      // texte(colonne, ligne, "LETTRES") écrit des lettres à l'écran.  
                      // L'écran fait 20 colonnes (0 à 19) sur 18 lignes (0 à 17).  
                      // Les lettres à écrire se mettent entre guillemets.  
                      // Écrit AVANT la boucle : une seule fois, et le A reste affiché.  
  
    while (true) { // La boucle de jeu : recommence pour toujours...  
        image(); // ... attendre l'image suivante (60 par seconde).  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un A, tout en haut à gauche de l'écran.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le A est dans la case (0, 0)
- ✓ la case d'à côté est vide

